Import pandas and numpy

In [1]:
```
import pandas as pd
import numpy as np
```

```
C:\Users\Prashant\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.
3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
```

Load data

In [2]:
```
data = pd.read_csv("train.csv")
data.head()
```

Out[2]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeat
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	N
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	N
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	N
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	N
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	N

5 rows × 81 columns

Data types

We can use pandas function dtypes to grab the data type of every column in a data frame.

In [3]:
```
data.dtypes  # data.info()   # columns----features in ML
```

Out[3]:
```
Id                int64
MSSubClass        int64
MSZoning         object
LotFrontage     float64
LotArea           int64
                  ...
MoSold            int64
YrSold            int64
SaleType         object
SaleCondition    object
SalePrice         int64
Length: 81, dtype: object
```

Alternatively, if we only want to consider a particular column, we can do this.

In [4]:
```
# Check the data type of the SalePrice column

data['SalePrice'].dtype
```

Out[4]:
```
dtype('int64')
```

What are the most common data types that you will see in pandas?

- int64 (integer)
- float64 (floating point number)
- object (string)
- datetime (datetime)
- bool (true or false)

We can convert a column of one type into another using the astype function.

In [5]:
```
# Convert the SalePrice column into float64 data type

data["SalePrice"]=data['SalePrice'].astype('float64')
```

In [6]:
```
data["SalePrice"]
```

Out[6]:
```
0       208500.0
1       181500.0
2       223500.0
3       140000.0
4       250000.0
          ...
1455    175000.0
1456    210000.0
1457    266500.0
1458    142125.0
1459    147500.0
Name: SalePrice, Length: 1460, dtype: float64
```

In [7]:
```
data.shape
```

Out[7]:
```
(1460, 81)
```

Locating missing values

First let's recall how we can figure out how many null values are there in our dataframe.

In [11]:
```
# How many null values are there in our dataframe?

data.isnull().sum()  #data.isna().sum()
```

Out[11]:
```
7829
```

Dealing with missing values

There are mainly two ways to deal with missing data.

1. Drop the rows or columns which contain missing data
2. Replace missing data with substituted values also known as imputation

Both methods have their own individual pros and cons. Which of the two methods you use will be highly dependent on your data as well as the nature of the problem you are trying to solve. If you are working on detailed piece of analysis, this is where you would take the time to really understand each column to figure out the best strategy to handle those missing values.

Generally speaking, dropping data is much easier and straightforward to implement but it does come at the expense of removing potentially useful information from our dataset. This will adversely affect model performance which then leads to inaccurate model predictions.

On the other hand, choosing the best way to impute or replace those missing values require more time, consideration and experience. I will briefly touch upon the different ways to impute missing values in the later part of this notebook.

In [ ]:
```
when rows has missing values like <5%---can drop

if a column has >25% missing----delete that
```

Method 1: Drop rows or columns with missing values

If you are in a hurry or don't have a reason to figure out why your values are missing, one option is to remove rows or columns that contain missing values. However, this is not the best approach in most cases because we might lose potentially useful information in our dataset.

Let's see how we can drop rows and columns with missing values using the dropna function.

In [64]:
```
# Drop rows with missing values

data.dropna()  #data.dropna(inplace=True)
```

Out[64]:

Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeatu

0 rows × 81 columns

In [ ]:
```
data
```

Yikes, it appears that we have dropped all the rows in our dataframe. This is not good.

Ideally, we would only remove rows if we have a large number of training examples and if the rows with missing data is not a high number. In our example, all the rows have at least one missing feature therefore dropping rows with missing data is not a good strategy to use.

Maybe we should remove columns with missing values instead.

In [28]:
```
# Drop columns with missing values

col_with_na_dropped = data.dropna(axis = 1)
col_with_na_dropped.head()
```

Out[28]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	LotShape	LandContour	Utilities	LotConfig	...	EnclosedPorch	3SsnPorch	Scre
0	1	60	RL	65.0	8450	Pave	Reg	Lvl	AllPub	Inside	...	0	0	
1	2	20	RL	80.0	9600	Pave	Reg	Lvl	AllPub	FR2	...	0	0	
2	3	60	RL	68.0	11250	Pave	IR1	Lvl	AllPub	Inside	...	0	0	
3	4	70	RL	60.0	9550	Pave	IR1	Lvl	AllPub	Corner	...	272	0	
4	5	60	RL	84.0	14260	Pave	IR1	Lvl	AllPub	FR2	...	0	0	

5 rows × 64 columns

In [29]:
```
# How much data did we lose?

print("Number of columns in original dataset: ", data.shape[1])  #(5,81)
print("Number of columns left after dropping: ", col_with_na_dropped.shape[1])
difference = data.shape[1] - col_with_na_dropped.shape[1]
print("We have dropped a total of %d columns." %difference)
```

```
Number of columns in original dataset:  81
Number of columns left after dropping:  64
We have dropped a total of 17 columns.
```

In [23]:
```
data.shape
```

Out[23]:
```
(1460, 81)
```

In [25]:
```
data.shape[1]
```

Out[25]:
```
81
```

In [31]:
```
col_with_na_dropped.shape[1]
```

Out[31]: 64

We are dropping a substantial amount of features from our dataset, almost a quarter!

Features in our example are the characteristics that describe the house. If we remove features that are significant in explaining the sale price of the house, our model will not be able to make accurate predictions.

In an ideal scenario, it is only safe to drop a column if there is significant random missing data present in a column and if we have reasons to believe that the column is unimportant in predicting our target variable.

Let's have a closer look at the features that we are dropping.

Method 2: Filling in missing values

There are a couple of ways to impute missing data that is subjective to the situation.

In this section, I will go through the two of the most common technique to fill missing data:

1. Using mean or median values (for numerical variables)
2. Using mode or zero (for categorical variables)

Numerical variables are continuous random variable like height, age, total sales whereas categorical variables are discrete random variables like yes or no, pass or fail, small, medium or large etc.

The main function to use here is the fillna function.

In [13]:
```
# Suppose we want to fill missing data in the LotFrontage column
# First let's examine the data type

data['LotFrontage'].mean()
```

Out[13]: 70.04995836802665

In [14]:
```
data['LotFrontage'].head(10)
```

Out[14]:
```
0    65.0
1    80.0
2    68.0
3    60.0
4    84.0
5    85.0
6    75.0
7     NaN
8    51.0
9    50.0
Name: LotFrontage, dtype: float64
```

Row number 8 has missing value.

Suppose we want to fill all missing data in that column with the median.

In [15]:
```
# Compute median

data['LotFrontage'].median()
```

Out[15]:
```
69.0
```

In [16]:
```
# Impute missing data in LotFrontage with median

data['LotFrontage'].fillna(data['LotFrontage'].median(),inplace=True)
data['LotFrontage'].head(10)
```

```
C:\Users\Prashant\AppData\Local\Temp\ipykernel_3892\809969695.py:3: FutureWarning: A value is trying to be set on a cop
y of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we
are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df
[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  data['LotFrontage'].fillna(data['LotFrontage'].median(),inplace=True)
```

Out[16]:
```
0    65.0
1    80.0
2    68.0
3    60.0
4    84.0
5    85.0
6    75.0
7    69.0
8    51.0
9    50.0
Name: LotFrontage, dtype: float64
```

Row number 8 has been filled with the median of the LotFrontage column that is 69.

Now let's look at an example of a categorical variable like GarageType.

In [17]:
```
# Check data type of GarageType column

data['GarageType'].dtype
```

Out[17]:
```
dtype('O')
```

In [18]:
```
# Let's see the value counts in that column including the nulll value

data['GarageType'].value_counts(dropna=False)
```

Out[18]:
```
GarageType
Attchd     870
Detchd     387
BuiltIn     88
NaN         81
Basment     19
CarPort      9
2Types       6
Name: count, dtype: int64
```

The most frequent observation is Attchd.

Suppose we want to fill the missing data with this observation.

In [35]:
```
data['GarageType'].mode()[0]
```

Out[35]:
```
'Attchd'
```

In [20]:
```
data['GarageType'].tail(10)
```

Out[20]:
```
1450        NaN
1451     Attchd
1452    Basment
1453        NaN
1454     Attchd
1455     Attchd
1456     Attchd
1457     Attchd
1458     Attchd
1459     Attchd
Name: GarageType, dtype: object
```

In [36]:
```
data.iloc[-10:,:]  #[-,-,-,-,-,-]
```

Out[36]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	Mi
1450	1451	90	RL	60.0	9000	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1451	1452	20	RL	78.0	9262	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1452	1453	180	RM	35.0	3675	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1453	1454	20	RL	90.0	17217	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1454	1455	20	FV	62.0	7500	Pave	Pave	Reg	Lvl	AllPub	...	0	NaN	NaN	
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	MnPrv	
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	GdPrv	
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	

10 rows × 81 columns

In [103…
```
data['GarageType'].fillna(data['GarageType'].mode()[0],inplace=True)
```

Out[103]:
```
1450     Attchd
1451     Attchd
1452    Basment
1453     Attchd
1454     Attchd
1455     Attchd
1456     Attchd
1457     Attchd
1458     Attchd
1459     Attchd
Name: GarageType, dtype: object
```

The missing values have now been replaced with the mode.

We can also fill the missing data with any number or text that we like. Let's consider the GarageQual feature.

Suppose we want to replace the null values with the word 'Unknown'.

In [21]:
```
data['GarageQual'].value_counts(dropna = False)
```

Out[21]:
```
GarageQual
TA     1311
NaN      81
Fa       48
Gd       14
Ex        3
Po        3
Name: count, dtype: int64
```

In [22]:
```
data['GarageQual'].fillna('Unknown',inplace=True)
data['GarageQual'].value_counts()
```

```
C:\Users\Prashant\AppData\Local\Temp\ipykernel_3892\1762493057.py:1: FutureWarning: A value is trying to be set on a co
py of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we
are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df
[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  data['GarageQual'].fillna('Unknown',inplace=True)
```

Out[22]:
```
GarageQual
TA         1311
Unknown      81
Fa           48
Gd           14
Ex            3
Po            3
Name: count, dtype: int64
```

Notice how the NaN value has been replaced with the word Unknown.

There are other more sophisticated methods of imputing missing data like using other features that are correlated to help determine the appropriate substitute value. However, I won't be covering those concepts in this tutorial but if you are interested, you can check out this article.